Seminario SQL Server 2005

Introduzione ai database temporali (parte 1)

Gianluca Hotz
gianluca@solidqualitylearning.com
ghotz@ugiss.org

Solid Quality Learning

Introduzione

Cos'é un database?

- Collezione strutturata di fatti
  - rappresenta proposizioni considerate vere (assiomi)
  - sottoinsieme di fatti del mondo reale di interesse
  - permette di derivare nuove proposizioni
    - tramite regole formali di inferenza

Introduzione

Proposizioni e predicato

- Database per dati relativi a prenotazioni
- Fatto (proposizione)
  - La prenotazione identificata dal numero **990000**, per la stanza **333**, prevede la data di arrivo **29/10/2006** e la data di partenza **01/11/2006**.
  - La prenotazione identificata dal numero **990001**, per la stanza **275**, prevede la data di arrivo **27/10/2006** e la data di partenza **01/11/2006**.
- Forma generalizzata (predicato)
  - La prenotazione identificata dal numero (**IDPrenotazione**), per la stanza (**NumStanza**), prevede la data di arrivo (**DataArrivo**) e la data di partenza (**DataPartenza**).

Introduzione

Basi del Modello Relazionale

- Concetto di relazione
  - non è la traduzione di *relationship* del modello E/R!
- Relazione matematica
  - sottoinsieme del prodotto cartesiano di domini
  - insieme di n-uple
    - (v1, v2, ..., vn) con v1 $\in$ D1, v2 $\in$ D2, …, vn $\in$ Dn
  - n-uple ordinate all'interno (stesso ordine dei domini)
- Relazione nel modello relazionale
  - n-uple (tuple) non sono ordinate all'interno
  - riferimento per nome (attributi)

Introduzione

Rappresentazione nel Modello Relazionale

- Consideriamo i domini
  - D1 = costituito da numeri interi
  - D2 = costituito da numeri interi
  - D3 = costituito da date
  - D4 = costituito da date
- Alcune possibili relazioni su D1, D2, D3, D4
  - {(990000, 333, 29/10/2006, 01/11/2006)}
  - {(990000, 333, 29/10/2006, 01/11/2006),
    (990001, 275, 27/10/2006, 30/10/2006)}

Introduzione
Relazioni e predicato

- Relazione rappresenta un predicato
  - interpretazione
    - di una porzione del mondo reale
    - di interesse
  - istanza del predicato
    - sostituendo le variabili con i valori delle tuple
    - otteniamo una proposizione
    - sempre vera per convenzione

Introduzione

Rappresentazione grafica nel Modello Relazionale

- Tramite tabelle
  - una tabella per ogni relazione (variabile relazione)
  - nomi degli attributi sono le intestazioni delle colonne
  - le colonne non sono ordinate
  - le righe sono le n-uple (tuple) di valori
  - le righe non sono ordinate
  - le righe sono distinte una dall'altra
    - in un insieme non possono essere presenti due valori uguali
    - una tabella rappresenta una relazione se le righe sono l'una diversa dall'altra

Introduzione

Esempio di rappresentazione grafica

- Predicato
  - La prenotazione identificata dal numero (**IDPrenotazione**), per la stanza (**NumStanza**), prevede la data di arrivo (**DataArrivo**) e la data di partenza (**DataPartenza**).
- Relazione
  - {(990000, 333, 29/10/2006, 01/11/2006), (990001, 275, 27/10/2006, 30/10/2006)}

IDPrenotazione	NumStanza	DataArrivo	DataPartenza
990000	333	2006-10-29	2006-11-01
990001	275	2006-10-27	2006-10-30

Introduzione

Esempi: fornitori e prodotti forniti

S (fornitore)

S#	SNAME	STATUS	CITY
S1	Gianluca	20	Varese
S2	Davide	10	Milano
S3	Andrea	30	Brescia
…	…	…	…

P (prodotto)

P#	PNAME
P1	Corso 1
P2	Corso 2
P3	Corso 3
P4	Corso 4
…	…

SP (prodotto fornito)

S#	P#
S1	P1
S1	P2
S2	P1
S2	P4
S3	P3
S3	P4
…	…

Database temporali
Introduzione

- Database non temporale (convenzionale)
  - codifica rappresentazione di proposizioni
  - contiene solo i dati correnti
  - dati modificati non appena le proposizioni cambiano
  - dati eliminati non appena le proposizioni cessano di essere vere
- Database temporale
  - codifica rappresentazione di proposizioni con timestamp
  - contiene anche dati storici
  - nessuna modifica o eliminazione diretta dei dati
  - sono oggetto di studio almeno dall'inizio degli anni 80

Database temporali

Proposizione con timestamp

S#	FROM
S1	1999-07-01

- Consideriamo la tupla, possibili interpretazioni
  - T1: il fornitore S1 ha attivato un contratto **il** 1 Luglio 1999
  - T2: il fornitore S1 ha un contratto attivo **dal** 1 Luglio 1999
  - T3: il fornitore S1 ha un contratto attivo **durante** tutto il periodo dal 1 Luglio 1999 ad oggi
- Sono tutte lecite
  - solitamente T2 e T3 sono equivalenti
  - alcune ambiguità
    - contratto prima del 1 Luglio 1999?
    - contratto valido per tutti i giorni dal 1 Luglio 1999?

Database temporali
Ambiguità proposizioni con timestamp

- T1: il fornitore S1 ha attivato un contratto il 1 Luglio 1999
  - non sappiamo nulla della storia precedente e successiva
- T2: il fornitore S1 ha un contratto attivo dal 1 Luglio 1999
  - solitamente non lo era prima (non necessariamente)
  - contratto valido tutti i giorni dalla data fino ad oggi
    - oggi è implicito, data di interrogazione
    - punto che si muove nel tempo
- T3: il fornitore S1 ha un contratto attivo durante tutto il periodo dal 1 Luglio 1999 ad oggi
  - solitamente non lo era prima (non necessariamente)
  - in questo caso diamo un intervallo preciso

Database temporali

Predicato con intervallo periodo di validità

- Predicato
  - T3: il fornitore <**S#**> ha un contratto attivo durante il periodo da <**b**> a <**e**>
    - è a contratto tutti i giorni tra **b** e **e**
    - in questo caso diamo un intervallo preciso
    - *valid time*
- Può essere utilizzato per
  - storico
  - contratti che terminano nel futuro
  - contratti che iniziano e terminano nel futuro

Database temporali

Time quanta

- Per non incorrere nei problemi del calcolo infinitesimale
  - la linea del tempo (timeline) è considerata come una sequenza finita, discreta e indivisibile di *time quanta*
  - l'intervallo contiene quindi un numero finito di *time quanta*
  - per praticità, consideriamo il *time quanta* come l'unità più piccola che può essere rappresentata nel database
  - in SQL Server accuratezza
    - 3.33 millisecondi (tipo dato datetime)
    - 1 minuto (tipo dato smalldatetime)

Database temporali

Time point

- *Time point* unità rilevante per soggetto in questione
  - per praticità considerato come indivisibile
  - per praticità consideriamo che abbia comunque una durata
  - nel resto degli esempi utilizzeremo il giorno
- Intervallo
  - inizia al *time point* **b** **e** finisce al *time point* **e**
  - può essere inteso come
    - l'insieme di tutti i *time point* **t** per cui **b <= t <= e**

Database temporali

Semitemporalizzazione

- Predicato S_SINCE
  - Dal giorno <**SINCE**> (e non il giorno immediatamente prima) il fornitore <**S#**> ha un contratto attivo.
- Predicato SP_SINCE
  - Dal giorno <**SINCE**> (e non il giorno immediatamente prima) il fornitore <**S#**> è in grado di fornire il prodotto **<P#>**.

S_SINCE

S#	SINCE
S1	d04
S2	d07
S3	d03
S4	d04
S5	d02

SP_SINCE

S#	P#	SINCE
S1	P1	d04
S1	P2	d05
S1	P3	d09
S1	P4	d05
S1	P5	d04
S1	P6	d06
S2	P1	d08
S2	P2	d09
S3	P2	d08
S4	P2	d06
S4	P4	d04
S4	P5	d05

Database temporali

Vincoli semitemporalizzazione (1)

- Chiave primaria non modificata
- Chiave esterna
  - dobbiamo tenere conto delle date

S_SINCE

S#	SINCE
S1	d04
S2	d07
S3	d03
S4	d04
S5	d02

SP_SINCE

S#	P#	SINCE
S1	P1	d04
S1	P2	d05
S1	P3	d09
S1	P4	d05
S1	P5	d04
S1	P6	d06
S2	P1	d08
S2	P2	d09
S3	P2	d08
S4	P2	d06
S4	P4	d04
S4	P5	d05

Database temporali

Vincoli semitemporalizzazione (2)

- Chiave primaria non modificata
- Chiave esterna
  - dobbiamo tenere conto delle date
  - esempio:
    - non possiamo dire che S4 è in grado di fornire il prodotto P1 a partire dal giorno d03 perché il contratto è iniziato il giorno d04

S_SINCE

S#	SINCE
S1	d04
S2	d07
S3	d03
S4	d04
S5	d02

SP_SINCE

S#	P#	SINCE
S1	P1	d04
S1	P2	d05
S1	P3	d09
S1	P4	d05
S1	P5	d04
S1	P6	d06
S2	P1	d08
S2	P2	d09
S3	P2	d08
S4	P2	d06
S4	P4	d04
S4	P5	d05
S4	P1	d03

Database temporali

Problemi semitemporalizzazione

- Storia
  - eliminazione della proposizione quando non è più vera
- Esempi
  - non sappiamo se S5 non fornisce ancora nessun prodotto oppure se lo ha fornito in passato e ora non lo fornisce più
  - se S5 ha smesso di fornire un prodotto, non sappiamo da quando ha smesso di fornirlo

S_SINCE

S#	SINCE
S1	d04
S2	d07
S3	d03
S4	d04
S5	d02

SP_SINCE

S#	P#	SINCE
S1	P1	d04
S1	P2	d05
S1	P3	d09
S1	P4	d05
S1	P5	d04
S1	P6	d06
S2	P1	d08
S2	P2	d09
S3	P2	d08
S4	P2	d06
S4	P4	d04
S4	P5	d05

Database temporali

Temporalizzazione completa (1)

- Predicato S_FROM_TO
  - Dal giorno <**FROM**> (e non il giorno immediatamente prima) fino al giorno <**TO**> (e non il giorno immediatamente dopo) il fornitore <**S#**> ha un contratto attivo.
- Predicato SP_FROM_TO
  - Dal giorno <**FROM**> (e non il giorno immediatamente prima) fino al giorno <**TO**> (e non il giorno immediatamente dopo) il fornitore <**S#**> è stato in grado di fornire il prodotto **<P#>**.

S_FROM_TO

S#	FROM	TO
S1	d04	d10
S2	d02	d04
S2	d07	d10
S3	d03	d10
S4	d04	d10
S5	d02	d10

SP_FROM_TO

S#	P#	FROM	TO
S1	P1	d04	d10
S1	P2	d05	d10
S1	P3	d09	d10
S1	P4	d05	d10
S1	P5	d04	d10
S1	P6	d06	d10
S2	P1	d02	d04
S2	P1	d08	d10
S2	P2	d03	d03
S2	P2	d09	d10
S3	P2	d09	d10
S4	P2	d06	d09
S4	P4	d04	d08
S4	P5	d05	d10

Database temporali

Temporalizzazione completa (2)

- Supponiamo che oggi sia d10
  - S2 ha avuto un contratto che è stato chiuso (valido da d02 a d04) e ne ha uno attivo valido da d07 ad oggi
  - S2 ha fornito il prodotto P1 per tutto il periodo del precedente contratto, poi ha ripreso a fornirlo con il nuovo contratto a partire da d08
  - S2 ha fornito il prodotto P2 solo il giorno d03 con il vecchio contratto ed ha ripreso a fornirlo a partire da d09 con quello nuovo
  - S4 ha fornito il prodotto P2 per il periodo da d06 a d09 con l'unico contratto attivo che è valido da d04 ad oggi
  - S5 non ha mai fornito alcun prodotto pur avendo un contratto attivo da d02 ad oggi

S_FROM_TO

S#	FROM	TO
S1	d04	d10
S2	d02	d04
S2	d07	d10
S3	d03	d10
S4	d04	d10
S5	d02	d10

SP_FROM_TO

S#	P#	FROM	TO
S1	P1	d04	d10
S1	P2	d05	d10
S1	P3	d09	d10
S1	P4	d05	d10
S1	P5	d04	d10
S1	P6	d06	d10
S2	P1	d02	d04
S2	P1	d08	d10
S2	P2	d03	d03
S2	P2	d09	d10
S3	P2	d09	d10
S4	P2	d06	d09
S4	P4	d04	d08
S4	P5	d05	d10

Database temporali

Vincoli temporalizzazione completa (1)

- Intervallo
  - **<TO>** deve essere maggior o uguale a **<FROM>**
- Chiave primaria
  - include attributo **<FROM>**
  - in alternativa attributo **<TO>**
  - esempio di chiavi candidate
  - scelta arbitraria

S_FROM_TO

S#	FROM	TO
S1	d04	d10
S2	d02	d04
S2	d07	d10
S3	d03	d10
S4	d04	d10
S5	d02	d10

SP_FROM_TO

S#	P#	FROM	TO
S1	P1	d04	d10
S1	P2	d05	d10
S1	P3	d09	d10
S1	P4	d05	d10
S1	P5	d04	d10
S1	P6	d06	d10
S2	P1	d02	d04
S2	P1	d08	d10
S2	P2	d03	d03
S2	P2	d09	d10
S3	P2	d09	d10
S4	P2	d06	d09
S4	P4	d04	d08
S4	P5	d05	d10

Database temporali

Vincoli temporalizzazione completa (2)

- Chiavi esterne
  - devono tenere conto degli intervalli

S_FROM_TO

S#	FROM	TO
S1	d04	d10
S2	d02	d04
S2	d07	d10
S3	d03	d10
S4	d04	d10
S5	d02	d10

SP_FROM_TO

S#	P#	FROM	TO
S1	P1	d04	d10
S1	P2	d05	d10
S1	P3	d09	d10
S1	P4	d05	d10
S1	P5	d04	d10
S1	P6	d06	d10
S2	P1	d02	d04
S2	P1	d08	d10
S2	P2	d03	d03
S2	P2	d09	d10
S3	P2	d09	d10
S4	P2	d06	d09
S4	P4	d04	d08
S4	P5	d05	d10

Database temporali

Vincoli temporalizzazione completa (3)

- Chiavi esterne
  - devono tenere conto degli intervalli
  - esempi:
    - non posso dire che S2 può fornire il prodotto P3 nell'intervallo tra d05 e d05 perché S2 non ha nessun contratto attivo nell'intervallo tra d04 e d06

S_FROM_TO

S#	FROM	TO
S1	d04	d10
S2	d02	d04
S2	d07	d10
S3	d03	d10
S4	d04	d10
S5	d02	d10

SP_FROM_TO

S#	P#	FROM	TO
S1	P1	d04	d10
S1	P2	d05	d10
S1	P3	d09	d10
S1	P4	d05	d10
S1	P5	d04	d10
S1	P6	d06	d10
S2	P1	d02	d04
S2	P1	d08	d10
S2	P2	d03	d03
S2	P2	d09	d10
S3	P2	d09	d10
S4	P2	d06	d09
S4	P4	d04	d08
S4	P5	d05	d10
S2	P3	d05	d06

Database temporali

Vincoli temporalizzazione completa (4)

- Chiavi esterne
  - devono tenere conto degli intervalli
  - esempi:
    - non posso dire che S2 può fornire il prodotto P3 nell'intervallo tra d05 e d05 perché S2 non ha nessun contratto attivo nell'intervallo tra d04 e d06
    - non posso dire che S4 può fornire il prodotto P1 nell'intervallo tra d02 e d10 perché il contratto inizia da d04

S_FROM_TO

S#	FROM	TO
S1	d04	d10
S2	d02	d04
S2	d07	d10
S3	d03	d10
S4	d04	d10
S5	d02	d10

SP_FROM_TO

S#	P#	FROM	TO
S1	P1	d04	d10
S1	P2	d05	d10
S1	P3	d09	d10
S1	P4	d05	d10
S1	P5	d04	d10
S1	P6	d06	d10
S2	P1	d02	d04
S2	P1	d08	d10
S2	P2	d03	d03
S2	P2	d09	d10
S3	P2	d09	d10
S4	P2	d06	d09
S4	P4	d04	d08
S4	P5	d05	d10
S4	P1	d02	d10

Database temporali

Vincoli temporalizzazione completa (5)

- Ulteriori vincoli per il predicato
  - non il giorno immediatamente prima dell'inizio e non il giorno immediatamente dopo la fine

S_FROM_TO

S#	FROM	TO
S1	d04	d10
S2	d02	d04
S2	d07	d10
S3	d03	d10
S4	d04	d10
S5	d02	d10

SP_FROM_TO

S#	P#	FROM	TO
S1	P1	d04	d10
S1	P2	d05	d10
S1	P3	d09	d10
S1	P4	d05	d10
S1	P5	d04	d10
S1	P6	d06	d10
S2	P1	d02	d04
S2	P1	d08	d10
S2	P2	d03	d03
S2	P2	d09	d10
S3	P2	d09	d10
S4	P2	d06	d09
S4	P4	d04	d08
S4	P5	d05	d10

Database temporali

Vincoli temporalizzazione completa (6)

- Ulteriori vincoli per il predicato
  - non il giorno immediatamente prima dell'inizio e non il giorno immediatamente dopo la fine
- Circonlocuzione
  - esempio
    - dicendo che il fornitore S4 ha un contratto attivo da d02 a d03 e da d04 a d10
    - giro di parole per dire che S4 ha un contratto attivo da d02 a d10

S_FROM_TO

S#	FROM	TO
S1	d04	d10
S2	d02	d04
S2	d07	d10
S3	d03	d10
S4	d04	d10
S5	d02	d10
S4	d02	d03

SP_FROM_TO

S#	P#	FROM	TO
S1	P1	d04	d10
S1	P2	d05	d10
S1	P3	d09	d10
S1	P4	d05	d10
S1	P5	d04	d10
S1	P6	d06	d10
S2	P1	d02	d04
S2	P1	d08	d10
S2	P2	d03	d03
S2	P2	d09	d10
S3	P2	d09	d10
S4	P2	d06	d09
S4	P4	d04	d08
S4	P5	d05	d10

Database temporali

Vincoli temporalizzazione completa (7)

- Ulteriori vincoli per il predicato
  - non il giorno immediatamente prima dell'inizio e non il giorno immediatamente dopo la fine
- Sovrapposizione
  - esempio
    - dicendo che il fornitore S1 ha un contratto attivo da d02 a d06 e da d04 a d10
    - duplicazione da d04 a d06
    - possibile errore
    - possibile accorpamento: S1 ha un contratto attivo da d02 a d10
  - alcune sovrapposizioni protette dalla chiave

S_FROM_TO

S#	FROM	TO
S1	d04	d10
S2	d02	d04
S2	d07	d10
S3	d03	d10
S4	d04	d10
S5	d02	d10
S1	d02	d06

SP_FROM_TO

S#	P#	FROM	TO
S1	P1	d04	d10
S1	P2	d05	d10
S1	P3	d09	d10
S1	P4	d05	d10
S1	P5	d04	d10
S1	P6	d06	d10
S2	P1	d02	d04
S2	P1	d08	d10
S2	P2	d03	d03
S2	P2	d09	d10
S3	P2	d09	d10
S4	P2	d06	d09
S4	P4	d04	d08
S4	P5	d05	d10

Database temporali

Problemi temporalizzazione completa

- Complessità
  - definizione vincoli
  - interrogazioni

S_FROM_TO

S#	FROM	TO
S1	d04	d10
S2	d02	d04
S2	d07	d10
S3	d03	d10
S4	d04	d10
S5	d02	d10

SP_FROM_TO

S#	P#	FROM	TO
S1	P1	d04	d10
S1	P2	d05	d10
S1	P3	d09	d10
S1	P4	d05	d10
S1	P5	d04	d10
S1	P6	d06	d10
S2	P1	d02	d04
S2	P1	d08	d10
S2	P2	d03	d03
S2	P2	d09	d10
S3	P2	d09	d10
S4	P2	d06	d09
S4	P4	d04	d08
S4	P5	d05	d10

Database temporali

Tipo dato Interval

- RDBMS deve supportare un type generator
  - funzioni accessorie
  - possibili rappresentazioni
  - è un tipo scalare, non ha componenti visibili all'utente
    - i punti di inizio e fine sono componenti delle possibili rappresentazioni, non dell'intervallo in quanto tale
- Notazione
  - b:e tra parentesi, sono *type point*
  - [] estremo incluso
  - () estremo incluso
  - esempio
    - [d04:d10], [d04:d11), (d03:d10], (d03:d11)
    - tutte possibili rappresentazioni dello stesso intervallo

Database temporali

Temporalizzazione completa con intervalli

- Predicato S_DURING
  - Dal giorno che è il punto di inizio di <**DURING**> (e non il giorno immediatamente prima) fino al giorno che è il punto di fine di <**DURING**> (e non il giorno immediatamente dopo) il fornitore <**S#**> ha un contratto attivo.

- Predicato SP_DURING
  - Dal giorno che è il punto di inizio di <**DURING**> (e non il giorno immediatamente prima) fino al giorno che è il punto di fine di <**DURING**> (e non il giorno immediatamente dopo) il fornitore <**S#**> è stato in grado di fornire il prodotto <**P#**>.

S_DURING

S#	DURING
S1	[d04:d10]
S2	[d02:d04]
S2	[d07:d10]
S3	[d03:d10]
S4	[d04:d10]
S5	[d02:d10]

SP_DURING

S#	P#	DURING
S1	P1	[d04:d10]
S1	P2	[d05:d10]
S1	P3	[d09:d10]
S1	P4	[d05:d10]
S1	P5	[d04:d10]
S1	P6	[d06:d10]
S2	P1	[d02:d04]
S2	P1	[d08:d10]
S2	P2	[d03:d03]
S2	P2	[d09:d10]
S3	P2	[d08:d10]
S4	P2	[d06:d09]
S4	P4	[d04:d08]
S4	P5	[d05:d10]

Database temporali

Benefici temporalizzazione con intervalli

- Scelta chiave primaria
  - solo una chiave candidata
  - nessuna scelta arbitraria
- Ambiguità intervalli aperti/chiusi
  - diverse possibili rappresentazioni
- Vincolo estremi dell'intervallo
  - inizio <= fine
  - incluso nella definizione del tipo
- Semplicità
  - vincoli
  - interrogazioni
  - ma servono operatori specifici!

S_DURING

S#	DURING
S1	[d04:d10]
S2	[d02:d04]
S2	[d07:d10]
S3	[d03:d10]
S4	[d04:d10]
S5	[d02:d10]

SP_DURING

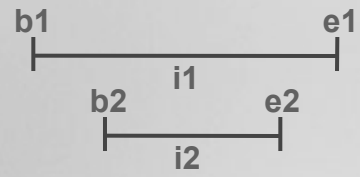
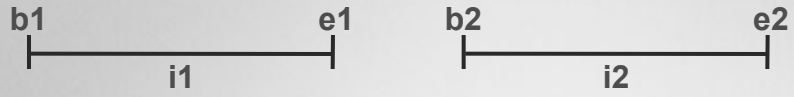
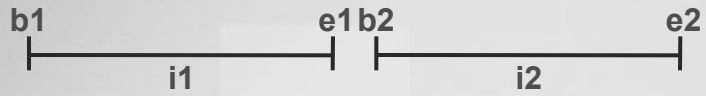
S#	P#	DURING
S1	P1	[d04:d10]
S1	P2	[d05:d10]
S1	P3	[d09:d10]
S1	P4	[d05:d10]
S1	P5	[d04:d10]
S1	P6	[d06:d10]
S2	P1	[d02:d04]
S2	P1	[d08:d10]
S2	P2	[d03:d03]
S2	P2	[d09:d10]
S3	P2	[d08:d10]
S4	P2	[d06:d09]
S4	P4	[d04:d08]
S4	P5	[d05:d10]

Database temporali
Operatori su intervalli

- Servono una serie di operatori per gli intervalli
  - basilari
    - BEGIN, END (*time point* iniziale/finale)
    - PRE, POST (*time point* precedente/successivo)
    - POINT FROM (time point dall'intervallo unitario)
    - appartenenza (time point appartiene a intervallo)
  - di confronto
  - altri operatori

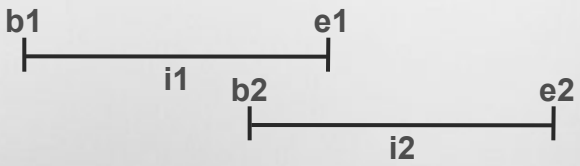
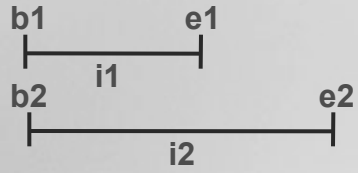
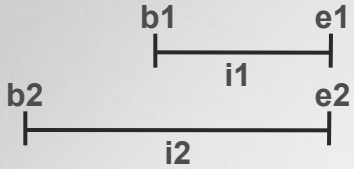
Database temporali

Operatori di confronto (1)

EQUALS		**b1=b2 AND e1=e2**
CONTAINS (DURING)		**b1<=b2 AND e1>=e2**
BEFORE (AFTER)		**e1<b2**
MEETS		**b2 = e1 + 1**

Database temporali

Operatori di confronto (2)

OVERLAPS		**b1<=e2 AND b2 <=e1**
BEGINS (STARTS)		**b1=b2 AND e1<=e2**
ENDS (FINISHES)		**e1=e2 AND b1>=b2**
MERGES	**OVERLAPS** OR **MEETS**	

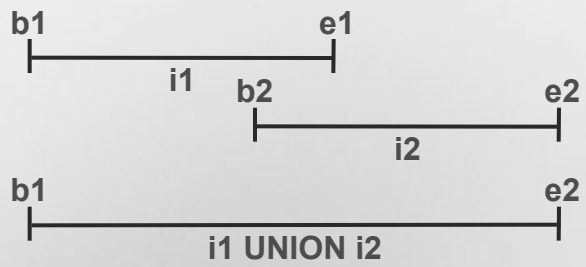
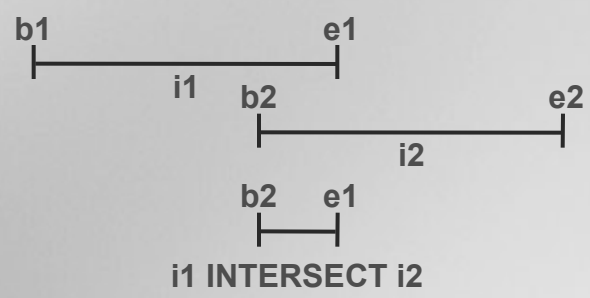
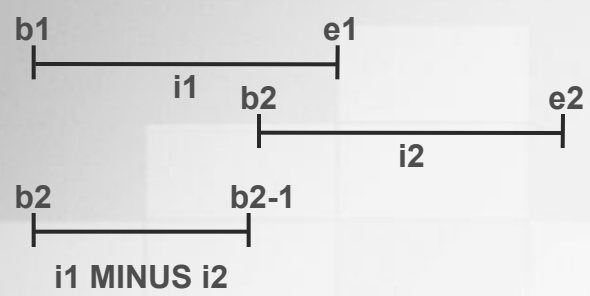
Database temporali

Altri operatori (1)

- COUNT (o DURATION)
  - numero di *time point* nell'intervallo

Database temporali

Altri operatori (2)

UNION		**[MIN(b1,b2):MAX(e1,e2)]** **se i1 MERGES i2 è vero altrimenti non è definito**
INTERSECT		**[MAX(b1,b2):MIN(e1,e2)]** **se i1 OVERLAPS i2 è vero altrimenti non è definito**
MINUS		**[b1:MIN(b2-1,e1)]** **se b1<b2 AND e1<=e2 è vero** **[MAX(e2+1):e1]** **se b1>=b2 AND e1>e2 è vero** **altrimenti non è definito**

Database temporali
Operatore EXPAND (UNFOLD)

- Si applica ad un insieme di intervalli (dello stesso tipo) e produce un insieme di intervalli (dello stesso tipo)
- Se X è un insieme di intervalli (dello stesso tipo), la sua forma “espansa” è l’insieme di tutti gli intervalli di tipo [p:p] dove p è un *type point* in qualche intervallo di X.
- Esempio
  - X1 = { [d01:d01], [d03:d05], [d04:d06] }
  - X2 = { [d01:d01], [d03:d04], [d05:d05], [d05:d06] }
  - X3 = { [d01:d01], [d03:d03], [d04:d04], [d05:d05], [d06:d06] }
  - X3 = EXPAND(X1) = EXPAND(X2)

Database temporali
Operatore COLLAPSE (COALESCE)

- Si applica ad un insieme di intervalli (dello stesso tipo) e produce un insieme di intervalli (dello stesso tipo)
- Se X è un insieme di intervalli (dello stesso tipo), la sua forma "collassata" è l'insieme Y di tutti gli intervalli dello stesso tipo per cui
  - X e Y hanno la stessa forma "espansa"
  - non esistono due intervalli distinti i1 e i2 di Y per i quali sia vera i1 MERGES i2 (in maniera equivalente: non esistono due intervalli distinti i1 e i2 di Y per i quali i1 UNION i2 sia definita)
- Esempio
  - X1 = { [d01:d01], [d03:d05], [d04:d06] }
  - X2 = { [d01:d01], [d03:d04], [d05:d05], [d05:d06] }
  - X3 = { [d01:d01], [d03:d03], [d04:d04], [d05:d05], [d06:d06] }
  - X4 = { [d01:d01], [d03:d06] }
  - X4 = COLLAPSE(X1) = COLLAPSE(X2) = COLLAPSE(X1)

Database temporali
Proprietà operatori EXPAND e COLLAPSE

- Proprietà
  - una forma espansa/collassata di X esiste sempre
  - la forma espansa/collassata è equivalente ad X
  - la forma espansa/collassata di X è univoca
  - se X ha cardinalità zero, anche il risultato ha cardinalità zero
  - se X ha cardinalità uno
    - il risultato di COLLAPSE ha cardinalità uno
    - il risultato di EXPAND ha cardinalità uno solo se l'unico intervallo di X è un intervallo unitario
  - COLLAPSE e EXPAND non sono uno l'inverso dell'altro
  - EXPAND(COLLAPSE(X)) = EXPAND(X)
  - COLLAPSE(EXPAND(X)) = COLLAPSE(X)

Database temporali

Generalizzazione degli operatori relazionali

- Tramite gli operatori EXPAND e COLLAPSE
  - due ulteriori operatori PACK/UNPACK per semplificare ma sono solo *shorthands*
- Per gestire intervalli di tempo (o generici)
  - union, intersection, difference
  - restriction, projection
  - join
  - extend, summarize
  - group, ungroup
  - relational comparison

Database temporali

Modello: solo dati correnti (1)

SSSC_SINCE

S#	SNAME	STATUS	CITY	SINCE
S1	Smith	20	London	d04

- Predicato SSSC_SINCE
  - Dal giorno <**SINCE**> (e non il giorno immediatamente prima) sono vere le seguenti:
    - il fornitore <**S#**> ha un contratto
    - il fornitore <**S#**> ha nome <**SNAME**>
    - il fornitore <**S#**> ha stato <**STATUS**>
    - il fornitore <**S#**> è situato nella città <**CITY**>

Database temporali

Modello: solo dati correnti (2)

SSSC_SINCE

S#	SNAME	STATUS	CITY	SINCE
S1	Smith	20	London	d04

SSSC_SINCE

S#	SNAME	STATUS	CITY	SINCE
S1	Smith	30	London	d10

- Problema
  - aggiorno lo stato del fornitore S1 a 30 il giorno d10
  - perdo alcune informazioni, esempio
    - che il fornitore era situato a Londra dal giorno d04
    - che il contratto con il fornitore è iniziato il giorno d04

Database temporali

Modello: solo dati correnti (3)

SSSC_SINCE

S#	S#_SINCE	SNAME	SNAME_SINCE	STATUS	STATUS_SINCE	CITY	CITY_SINCE
S1	d04	Smith	d04	20	d04	London	d04

- Predicato SSSC_SINCE
  - Il fornitore <**S#**> è sotto contratto dal giorno <**S#_SINCE**>, ha nome <**SNAME**> dal giorno <**SNAME_SINCE**>, ha stato <**STATUS**> dal giorno <**STATUS_SINCE**> ed è situato della città <**CITY**> dal giorno <**CITY_SINCE**>
  - Tenendo presente che per tutti i giorni “since” si intende sempre il giorno stesso e non il giorno immediatamente prima

Database temporali

Modello: solo dati correnti (4)

SSSC_SINCE

S#	S#_SINCE	SNAME	SNAME_SINCE	STATUS	STATUS_SINCE	CITY	CITY_SINCE
S1	d04	Smith	d04	30	d10	London	d04

SSSC_SINCE

S#	S#_SINCE	SNAME	SNAME_SINCE	STATUS	STATUS_SINCE	CITY	CITY_SINCE
S1	d04	Smith	d04	20	d04	Paris	d10

- Problemi precedenti risolti
- Ci sono altri problemi
  - aggiorno la città dove è situato il fornitore S1 con Parigi il giorno d10
  - perdo l'informazione che era situato a Londra (e quando)
  - in generale questo modello non mi permette di tenere la storia

Database temporali

Modello: solo dati storici (1)

SSSC_DURING

S#	SNAME	STATUS	CITY	DURING
S2	Jones	10	Paris	[d02:d04]

- Predicato SSSC_DURING
  - Dal giorno che è il punto di inizio di <**DURING**> (e non il giorno immediatamente prima) fino al giorno che è il punto di fine di <**DURING**> (e non il giorno immediatamente dopo) sono vere le seguenti:
    - il fornitore <**S#**> ha un contratto
    - il fornitore <**S#**> ha nome <**SNAME**>
    - il fornitore <**S#**> ha stato <**STATUS**>
    - il fornitore <**S#**> è situato nella città <**CITY**>

Database temporali

Modello: solo dati storici (2)

SSSC_DURING

S#	SNAME	STATUS	CITY	DURING
S2	Jones	10	Paris	[d02:d04]

SSSC_DURING

S#	SNAME	STATUS	CITY	DURING
S2	Jones	10	Paris	[d02:d03]
S2	Jones	15	Paris	[d04:d04]

- Problema
  - veniamo a conoscenza del fatto che il fornitore S2 ha cambiato stato a 15 il giorno d04
  - aggiornamento non intuitivo
  - valori degli altri attributi ripetuti
  - intervallo copre troppi attributi

Database temporali

Modello: solo dati storici (4)

S_DURING

S#	DURING
S2	[d02:d04]

S_NAME_DURING

S#	SNAME	DURING
S2	Jones	[d02:d04]

S_CITY_DURING

S#	CITY	DURING
S2	Paris	[d02:d04]

S_STATUS_DURING

S#	STATUS	DURING
S2	10	[d02:d03]
S2	15	[d04:d04]

- Decomposizione verticale
- Ancora un problema
  - *The Moving Point Now*

Database temporali

The Moving Point Now

- Fino ad ora abbiamo assunto che gli intervalli
  - iniziano un giorno ben preciso
  - terminano un giorno ben preciso
    - nel futuro (se effettivamente è così)
    - “ora”, al momento della richiesta (se non sappiamo quando)
    - se non sappiamo quando, abbiamo usato valori per intendere “ora”
- Proposte
  - utilizzo di una funzione “NOW”
    - molti problemi a livello concettuale
  - utilizzo *time point* massimo “END(during)”
    - utilizzato frequentemente
    - l’utente deve conoscerne il significato
    - bisogna tenerne conto in alcune operazioni
    - le proposizioni non sono più strettamente vere

Database temporali

Modello: ibrido dati storici e correnti

SSSC_SINCE

S#	S#_SINCE	SNAME	SNAME_SINCE	STATUS	STATUS_SINCE	CITY	CITY_SINCE
S2	d02	Jones	d02	15	d04	Paris	d02

S_DURING

S#	DURING

S_NAME_DURING

S#	SNAME	DURING

S_CITY_DURING

S#	CITY	DURING

S_STATUS_DURING

S#	STATUS	DURING
S2	10	[d02:d03]

- Decomposizione orizzontale e verticale
- Complicazione
  - vincoli, richieste e aggiornamenti

Database temporali

Prossime sessioni

- Vincoli e temporalità
- Stated times e Logged times
  - aka valid times e transaction times
  - riproducibilità
- Supporto ANSI SQL
- Implementazione in T-SQL
- Altro

Database Temporali

Risorse

- Temporal Data and The Relational Model
  C.J.Date, Hugh Darwen, Nikos A. Lorentzos
  ISBN 1-55860-855-9

Introduzione ai database temporali

Risorse generiche

- **Presentazione e demo**
  http://blogs.ugidotnet.org/ghotz
- **Informazioni sul prodotto SQL Server**
  http://www.microsoft.com/italy/sql
  http://www.microsoft.com/sql/2005
- **Supporto sviluppo ed professionisti IT**
  http://www.microsoft.com/technet/prodtechnol/sql
  http://msdn.microsoft.com/sql
- **Comunità SQL Server**
  http://www.ugiss.org
  http://www.microsoft.com/sql/community
- **Newsgroups pubblici via NTTP**
  microsoft.public.it.sql
  microsoft.public.sqlserver.*
- **Newsgroups pubblici via WEB**
  http://www.microsoft.com/italy/communities/newsgroups/default.mspx
  http://www.microsoft.com/technet/community/newsgroups/server/sql.mspx
- **Formazione e mentoring**
  http://www.microsoft.com/italy/traincert
  http://www.solidqualitylearning.com/

Attribution-NonCommercial-ShareAlike 2.5 - http://creativecommons.org/licenses/by-nc-sa/2.5/